Microelectronics Journal 161 (2025) 106697



Contents lists available at ScienceDirect

Microelectronics Journal

journal homepage: www.elsevier.com/locate/mejo





A RISC-V based SoC for blockchain data integration in IoT edge devices

Wentao Xu [a,*], Xiang Li [a], Qiuyan Xu [a], Yuxuan Zhang [a], Xiang Wu [a], Weilin Li [a], Ruize Wang [a], Gang Liang [a], Hao Guo [b,**]

[a] School of Microelectronics, Northwestern Polytechnical University, Xi'an, 710072, China
[b] School of Software, Northwestern Polytechnical University, Xi'an, 710072, China

ARTICLE INFO

Keywords:
Blockchain
Internet of things
RISC-V
Secure coprocessor
SoC
Trusted execution environment

ABSTRACT

The widespread growth of the Internet of Things (IoT) has significantly increased the need for robust data interaction mechanisms, making data security a critical challenge. Blockchain technology, characterized by its decentralized architecture, presents an innovative solution for managing data within IoT ecosystems. A collaborative cloud-edge framework emerges as a dependable option for deploying IoT blockchains; however, energy-efficient hardware support on the edge remains insufficient. To resolve this issue, this study introduces a low-power System on Chip (SoC) integrated with a specialized secure coprocessor to handle data on-chain processes. To mitigate physical-level security risks, the SoC incorporates a trusted computing architecture based on dual RISC-V cores. Experimental results using an FPGA platform reveal that the proposed SoC achieves a 12.1-fold performance enhancement for complete data on-chain processing tasks compared to the high-performance Intel i9-13950HX CPU, with total power consumption limited to just 0.579 W.

1. Introduction

The Internet of Things (IoT), a key part of Industry 4.0 [1], acts as a crucial link between the digital and physical realms [2,3], significantly shaping modern lifestyles. With the rapid evolution of IoT technologies, the global adoption of smart devices is skyrocketing. By 2025, the number of IoT devices is anticipated to reach 38.6 billion, escalating further to 50 billion by 2030 [4]. These devices, interconnected via wireless networks, constantly engage with their surroundings, producing immense volumes of diverse data resources across various sectors, including healthcare, smart homes, and manufacturing [5]. A substantial portion of this data is highly sensitive, yet current IoT systems lack robust measures to guarantee data integrity, confidentiality, and authenticity. Blockchain technology, first introduced through Satoshi Nakamoto's Bitcoin project in 2008, remains its most prominent and successful application to date [6,7]. Over the years, the use of blockchain has expanded beyond cryptocurrencies, garnering significant interest across a variety of industries [8]. Unlike conventional databases, which depend on centralized data management, blockchain operates as a distributed ledger technology [9], allowing multiple parties to collaboratively manage shared data. This approach provides key advantages such as traceability, immutability, and decentralization [10]. By integrating blockchain's distributed architecture into IoT systems as a replacement for traditional centralized frameworks, critical data security challenges can be effectively mitigated. As a result, numerous studies have investigated the integration of blockchain technology within IoT systems. In this context, Dorri et al. [11] proposed a lightweight blockchain architecture specifically designed for IoT, reducing the overhead of traditional blockchain systems while preserving security and privacy. Similarly, Samaniego et al. [12] explored a blockchain-as-a-service platform for IoT, utilizing cloud and fog computing to support lightweight implementations. Chen et al. [13] presented a blockchain-based data inspection scheme aimed at protecting data integrity in IoT environments. Additionally, Dorri et al. [14] developed a blockchain framework for smart home applications, emphasizing the elimination of proof-of-work mechanisms to lower implementation costs. Despite these advancements, most of these studies concentrate on improving protocols and optimizing software algorithms, with insufficient attention given to hardware considerations. Building IoT blockchains exclusively on servers poses several significant challenges. Firstly, the computational demands of processing large-scale data on-chain place considerable strain on server performance. As the number of IoT devices grows rapidly, this approach leads to escalating deployment costs, creating barriers to broader

* Corresponding author.
** Corresponding author.
E-mail addresses: xuwentao@nwpu.edu.cn (W. Xu), haoguo@nwpu.edu.cn (H. Guo).

https://doi.org/10.1016/j.mejo.2025.106697
Received 6 February 2025; Received in revised form 7 April 2025; Accepted 20 April 2025
Available online 24 April 2025
1879-2391/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

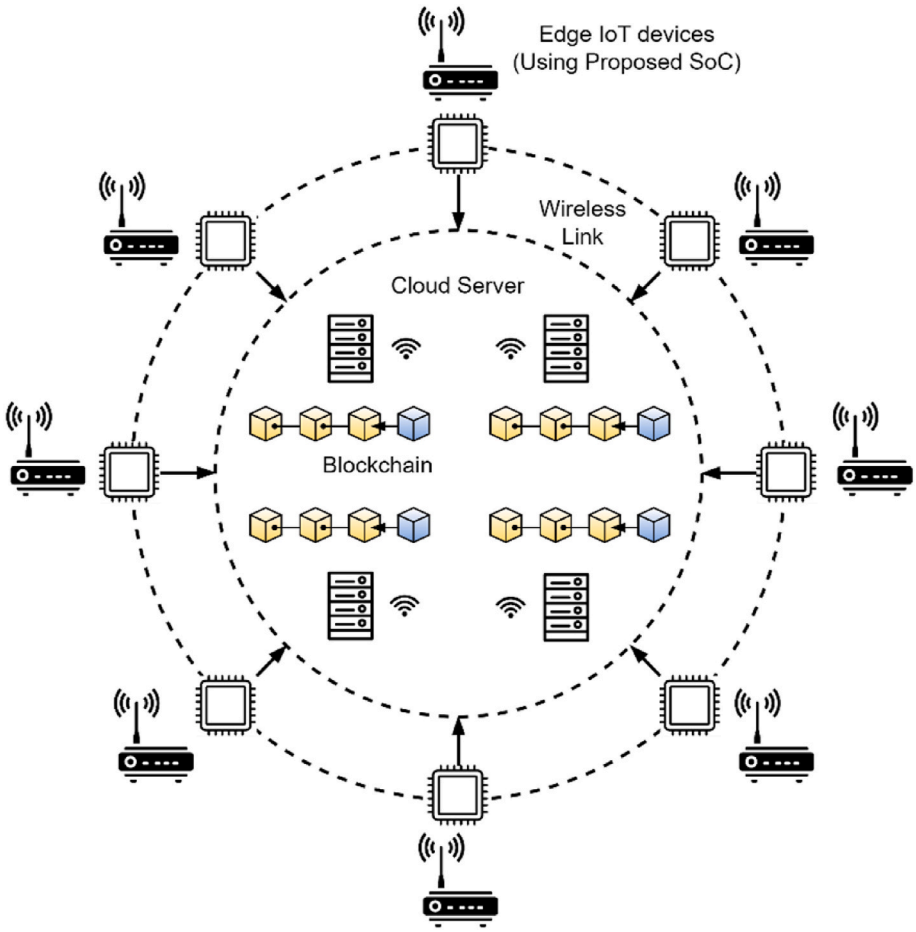


Fig. 1. Deploying IoT blockchain through cloud-edge collaborative architecture.

implementation. Moreover, data transmitted between terminal devices and servers over unsecured channels exposes sensitive information to potential security threats. A cloud-edge collaborative model offers a more practical alternative by assigning pre-on-chain processing tasks to edge IoT devices, while servers focus on tasks such as verification, block packaging, and data storage.

IoT terminals, which are optimized for low power consumption and portability, are generally resource-limited embedded systems with limited computational and storage capacities. This makes them unsuitable for managing the resource-intensive tasks associated with blockchain operations. To address this shortcoming, various accelerator designs have been developed that exploit hardware-level parallelism to enhance the computational efficiency of blockchain-related algorithms. For instance, Zhang et al. [15] utilized pipelining techniques to develop SHA-256 accelerators, significantly improving data throughput. Algredo-Badillo et al. [16] revealed that the sequence of operations within each iteration of the SHA-256 algorithm reduces the critical path length. Huang et al. [17] proposed an FFT-based modular multiplication and exponentiation algorithm, accompanied by corresponding hardware architectures, to optimize RSA cryptosystems with extended key lengths. Ghoreishi et al. [18] improved RSA computational efficiency by refining modular multiplication algorithms and introducing optimized hardware designs. Hentabli et al. [19] designed an RSA encryption unit leveraging a systolic array architecture for enhanced performance. While these designs have achieved significant performance improvements, they primarily target individual algorithms. However, the data-on-chain process necessitates the integration and coordination of multiple algorithms. Implementing multiple standalone accelerators on edge devices simultaneously reduces hardware integration efficiency and flexibility. Additionally, it introduces data transfer bottlenecks between accelerators, adversely affecting overall system performance. To overcome these challenges, this study presents a secure coprocessor designed for data-on-chain operations, integrated with a processor soft core within a System-on-chip (SoC) [20]. Utilizing a hardware-software co-design approach, this solution addresses computational constraints while maintaining system flexibility and control. RISC-V, an open-source Reduced Instruction Set Computing (RISC) architecture, is an excellent option for developing customized SoCs in IoT applications. Its open-source design eliminates the dependency on IP vendors and enables designers to design the microarchitecture to meet specific needs [21].

Additionally, the security of the hardware is equally critical. In the blockchain deployment model proposed in this paper, numerous essential operations for data-on-chain processing are handled by custom SoCs within edge IoT devices. However, these devices are frequently deployed in uncontrolled physical environments, exposing them to potential attacks. An attacker could directly target these devices to steal cryptographic keys, manipulate or falsify critical data, or even gain full control of the device, which could disturb the integrity and functionality of the entire blockchain system. To address these concerns, this design incorporates a Trusted Execution Environment (TEE) within the SoC. By utilizing hardware isolation, sensitive operations are executed within a secure enclave, enhancing the overall system security. The key contributions of this paper are outlined as follows:

(1) **Custom RISC-V Based SoC Design:** Designed to the specific needs of IoT and the operational requirements of blockchain, a custom SoC was developed featuring a dedicated coprocessor.

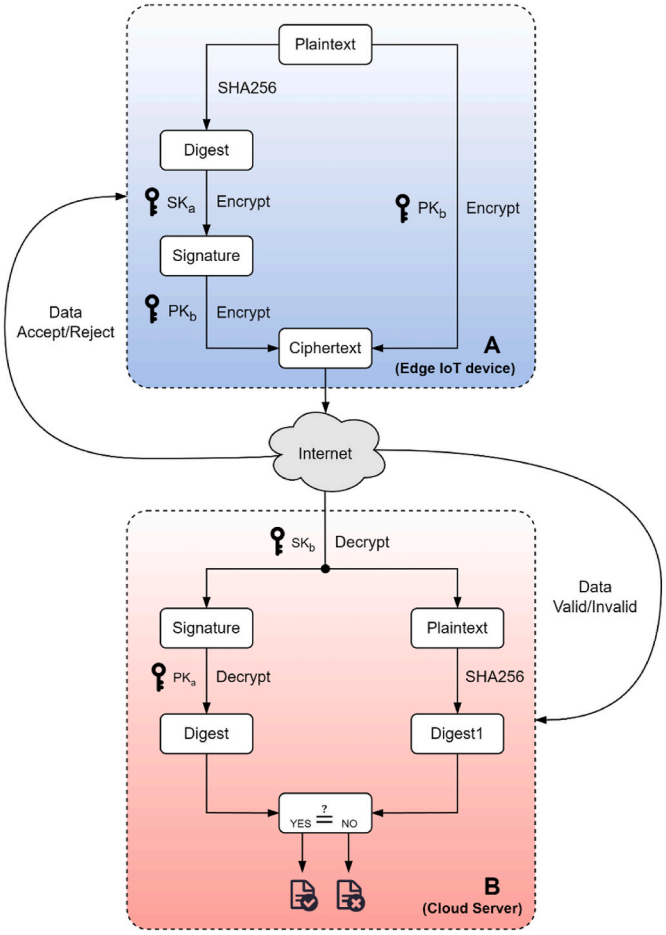


Fig. 2. Data processing and verification process – a peer-to-peer transmission example.

This design offers edge hardware support for building IoT blockchains through a cloud-edge collaborative model.

(2) **Secure Coprocessor:** The secure coprocessor proposed in this design is specifically optimized for data-on-chain processing, incorporating components such as a random number generator, SHA-256 computation unit, and RSA encryption engine. Each component has been fine-tuned at the circuit design level to ensure optimal performance while minimizing power consumption. FPGA platform testing demonstrates that for complete data-on-chain operations, the proposed SoC delivers a 12.1x performance improvement over the high-performance Intel i9-13950HX CPU, with static power consumption of 0.164 W and dynamic power consumption of 0.416 W.

(3) **Trusted Computing Architecture:** A trusted computing architecture is introduced for the SoC, featuring a dual-core design and a physical memory protection unit. This setup allows user programs and secure programs to operate independently on separate cores, ensuring that user programs cannot access sensitive, protected data.

The structure of this paper is organized as follows: Section II presents the research background. Section III outlines the design of the SoC. Section IV discusses the evaluation of experimental results. Finally, Section V concludes with a summary of the findings.

2. Background

2.1. Data-on-chain process

As the name suggests, a blockchain is a chain-like data structure made up of linked blocks, where each block includes the hash value of the previous block, ensuring continuity between neighboring blocks [22]. Each block is composed of two primary parts: the block header and the block body. The block header holds key information, such as the block's hash, the Merkle root, and the timestamp, while the block body contains various sensitive data [23,24]. Once critical data is generated, it is transmitted from its source to devices with specific permissions for block packaging and on-chain integration. To protect against data leakage during transmission and to authenticate the sender's identity, the data is preprocessed, which usually involves hashing, encryption, and digital signing [25]. In IoT systems, data intended for the blockchain primarily comes from edge devices. Fig. 1 depicts the architecture of an IoT blockchain within a cloud-edge collaborative framework, based on a consortium blockchain model. The outer layer is made up of edge devices equipped with custom SoCs, spread across different geographical locations. The inner layer consists of a network of member nodes, which are servers that hold the authority to access and modify the blockchain. Edge devices, located close to the data sources, process the collected data in real-time. The processed data is subsequently transmitted to the servers using lightweight wireless transmission protocols for secure validation. Once the data is validated, the servers package it into a block, add the block to the blockchain, and broadcast the newly added block to other devices within the node network. Fig. 2 illustrates an example of the data processing and validation workflow in a cloud-edge communication scenario. In this scenario, A represents an edge device, and B represents a cloud server. The diagram outlines the steps involved in securely transferring and validating data between the two endpoints within the blockchain network.

In **A**, the data processing includes the following four steps:

(1) **Digest Generation:** The SHA-256 algorithm is used to convert plaintext data into a hash value, known as the *Digest*. This algorithm maps data of any size to a fixed 256-bit hash value, and the process is irreversible [26]. The resulting hash value ensures the uniqueness of the plaintext, as even the smallest change in the data will produce a significantly different hash, thereby safeguarding data integrity.

(2) **Digital Signature:** This step utilizes asymmetric encryption algorithms [27], such as the RSA algorithm. Asymmetric encryption employs a pair of public and private keys for encryption and decryption. The private key is securely stored by its owner and must remain confidential, while the public key can be shared openly. Device A signs the hash value generated in the previous step with its private key, K_a. This signature authenticates the data's origin, as only the device possessing the private key can produce the signature.

(3) **Data Encryption:** In this step, both the plaintext and the digital signature are encrypted using the recipient's public key, PK_b, to produce ciphertext. Only device B, which possesses the corresponding private key SK_b, can decrypt this ciphertext. This ensures that the data remains secure and is protected from interception or tampering during transmission.

(4) **Data Transmission:** Device **A** sends the generated ciphertext over the internet to device B for further on-chain processing.

Upon receiving the data, device **B** performs the following three steps to validate it before adding it to the blockchain:

(1) **Decrypt Data:** Device B uses its private key SK_b to decrypt the ciphertext, extracting the plaintext and digital signature.

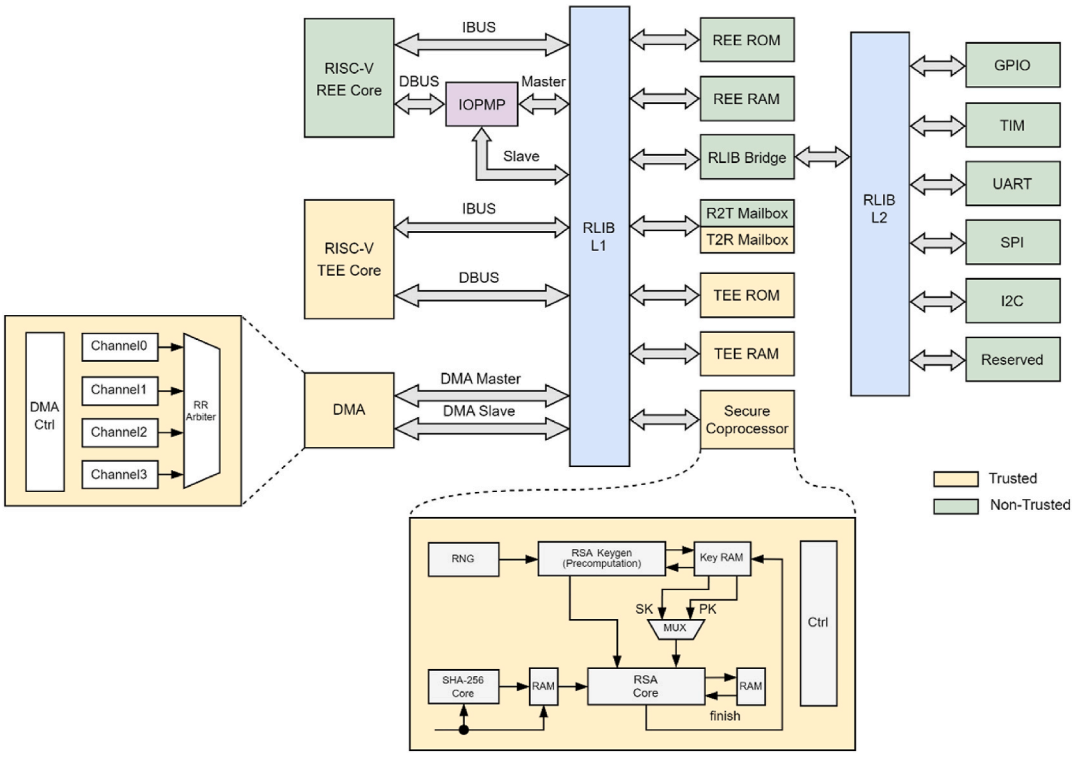


Fig. 3. The architecture of the proposed SoC.

(2) **Recompute Digest:** The hash value of the plaintext is recalculated, referred to as *Digest*1.
(3) **Identity and Data Integrity Verification:** Using device A's public key, PK_a, device B verifies the signature to recover the original hash value, *Digest*, that was generated by device A. If *Digest* matches *Digest*1, the validation is successful. Otherwise, validation fails, and device B rejects the data, discarding it instead of adding it to the blockchain.

Device A and device B ensure the reliability of data transmission through a handshake mechanism. During ciphertext transmission from device A, a data validity flag is simultaneously transmitted to notify device B of pending data requiring processing. Correspondingly, upon completing verification procedures, device B transmits a verification result message to device A regardless of authentication outcome. Subsequent transmission cycles from device A will be initiated only after confirming receipt of this acknowledgment.

2.2. SHA-256 algorithm

This section explains the SHA-256 algorithm. The SHA-256 algorithm processes a fixed 512-bit data block and generates a 256-bit hash value as output. To make the input length compatible, the message is padded before processing: first, a single bit of 1 is appended to the message. Next, multiple 0 bits are added until the message length, modulo 512, equals 448. Lastly, a 64-bit representation of the original message length is appended. This padding ensures that the total message length becomes a multiple of 512 bits. The padded message is then divided into 512-bit blocks, and each block is processed sequentially.

Each 512-bit message block M_i is split into 16 32-bit words, labeled as $W_0, W_1, \ldots, W_{15}$. These initial words are then expanded to 64 32-bit words, $W_{16}, W_{17}, \ldots, W_{63}$, using the recursive Equation (1).

$$W_t = \sigma_1(W_{t-2}) + W_{t-7} + \sigma_0(W_{t-15}) + W_{t-16} \tag{1}$$

Where σ_0 and σ_1 are defined as follows:

$$\sigma_0(x) = S^7(x) \oplus S^{18}(x) \oplus R^3(x) \tag{2}$$

$$\sigma_1(x) = S^{17}(x) \oplus S^{19}(x) \oplus R^{10}(x) \tag{3}$$

S^n, R^n, and n represent the circular right shift function, logical right shift function, and the number of bits to shift, respectively.

A message compression is then performed. The SHA-256 algorithm defines eight 32-bit initial hash values for the first message block and 64 32-bit constants, denoted as $H_0(0), H_1(0), \ldots, H_7(0)$ and $K_0, K_1, \ldots, K_{63}$, respectively. The former serves as the initial values for the eight state variables $A \sim H$, while the latter is used in iterative computations. The core compression function of SHA-256 involves 64 iterative rounds. During each round, a constant K_t and an expanded word W_t are utilized to update the eight state variables. The update process is governed by Equations (4) and (5).

$$T_1 = H + \Sigma_1(E) + Ch(E,F,G) + K_t + W_t \tag{4}$$

$$T_2 = \Sigma_0(A) + Maj(A,B,C) \tag{5}$$

Where the functions Σ_0, Σ_1, *Ch*, and *Maj* are defined as follows:

$$\Sigma_0(x) = S^2(x) \oplus S^{13}(x) \oplus S^{22}(x) \tag{6}$$

$$\Sigma_1(x) = S^6(x) \oplus S^{11}(x) \oplus S^{25}(x) \tag{7}$$

$$Ch(x,y,z) = (x \wedge y) \oplus (\neg x \wedge z) \tag{8}$$

$$Maj(x,y,z) = (x \wedge y) \oplus (x \wedge z) \oplus (y \wedge z) \tag{9}$$

At the end of each round, the state variables are updated using Equation (10).

$$A = T_1 + T_2, B = A, C = B, D = C, E = D + T_1, F = E, G = F, H = G \tag{10}$$

After completing the 64 rounds of compression, the state variables $A \sim H$ are added to the initial hash values $H_0(0)$, $H_1(0)$, …, $H_7(0)$,

generating a new set of intermediate hash values $H_0(1), H_1(1), \ldots, H_7(1)$). These intermediate hash values serve as the initial values for processing the next message block. Once all message blocks have been processed through these operations, the final output is derived by concatenating the resulting eight 32-bit hash values.

2.3. RSA algorithm

RSA encryption is another fundamental technique employed to ensure the security of blockchain data, offering valuable insights for developing hardware acceleration units. As one of the most established asymmetric encryption methods, RSA is highly compatible with a wide range of systems [28]. This algorithm serves dual purposes: encryption and digital signature creation. While the procedures for both tasks are largely the same, they differ primarily in the keys utilized.

RSA operations begin with the generation of a key pair. This process involves selecting two large prime numbers, p and q, at random. For robust security, these primes should have a bit length of at least 512 bits. The modulus n, which forms a critical component of both the public and private keys, is then calculated using p and q as shown in Equation (11).

$$n = p \times q \tag{11}$$

Next, the Euler's totient function of n, represented as $\varphi(n)$, is defined as follows:

$$\varphi(n) = (p-1) \times (q-1) \tag{12}$$

A random integer is selected as the public key exponent e, ensuring that $1 < e < \varphi(n)$ and e is coprime with $\varphi(n)$. The commonly selected value for e is 65537. Subsequently, the modular multiplicative inverse of e with respect to $\varphi(n)$, denoted as d, is calculated such that e and d satisfy the relationship described in Equation (13). The value d then serves as the private key exponent.

$$ed \equiv 1 (mod\ \varphi(n)) \tag{13}$$

Finally, the public key is (n, e), while the private key is (n, d).

If the plaintext is denoted by m, the sender encrypts it using the recipient's public key as described in Equation (14). The recipient, in turn, decrypts the ciphertext c using their private key according to Equation (15).

$$c \equiv m^e\ mod\ n \tag{14}$$

$$m \equiv c^d\ mod\ n \tag{15}$$

3. Proposed system

3.1. Overview of the SoC architecture

Fig. 3 depicts the architecture of the SoC designed for data-on-chain processing. This design incorporates RISC-V cores, a custom bus system, on-chip memory, multiple peripherals, and a dedicated security coprocessor. The SoC integrates two RISC-V cores, each built with a custom three-stage pipeline Harvard architecture supporting the RV32IM instruction set. These cores are specialized: the Rich Execution Environment (REE) core manages general-purpose tasks, while the Trusted Execution Environment (TEE) core focuses exclusively on blockchain security operations. Addressing low-power requirements, the SoC replaces traditional AHB or AXI buses with a custom-designed RISC-V Lightweight Internal Bus (RLIB). The RLIB protocol is simple and adaptable, operating with two independent channels command and return for shared read and write operations. The bus employs a fixed-priority arbitration mechanism and supports an arbitrary number of master-slave connections. Different masters can communicate with their respective slaves simultaneously, ensuring that the TEE and REE cores can execute distinct programs in parallel. The SoC employs a hierarchical two-level RLIB bus structure. The Level 1 (L1) bus connects key components such as memory, inter-core communication modules, and trusted peripherals. Meanwhile, the Level 2 (L2) bus, linked to the L1 bus via a synchronization bridge, accommodates general-purpose peripherals like GPIO and UART. For enhanced security and functional clarity, the REE and TEE cores are assigned separate memory spaces. An Input/Output Physical Memory Protection (IOPMP) Unit enforces strict access controls, ensuring the REE core can interact only with general peripherals, while trusted peripherals remain accessible exclusively to the TEE core. A DMA module, designed for the security coprocessor and supporting up to four channels, utilizes a polling arbitration mechanism and is connected to both the master and slave interfaces of the L1 bus, ensuring high-bandwidth memory access and eliminating potential bottlenecks in data handling.

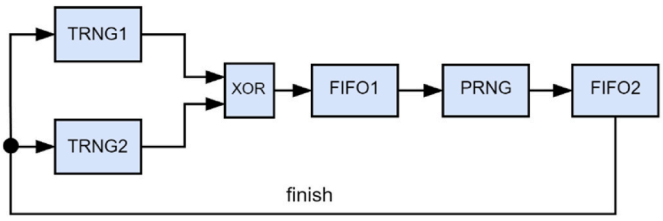


Fig. 4. Structure of the random number generator.

3.2. Security coprocessor

The security coprocessor, a critical element of the SoC, is designed to facilitate efficient data-on-chain processing. Its architecture integrates a random number generator, a dedicated SHA-256 computation unit, an RSA key generation module, an RSA encryption engine, and a secure key storage unit. Keys generated locally by the hardware and the public keys of external devices are stored in the key storage unit. A control module directs the operation of a multiplexer to feed keys into the RSA encryption engine, enabling encryption or digital signature processes. Once computations are finalized, the encryption engine activates a flag signal that instructs the key storage unit to erase the used keys, significantly mitigating the risk of key exposure. The microarchitecture of this coprocessor is depicted in Fig. 3, highlighting its functional components and operational workflow.

3.2.1. Random number generator

The key generation module relies on random numbers to produce key pairs, making the quality of randomness critical for system security. Traditional software-based random number generation employs pseudorandom algorithms, where the output depends on a seed value derived from factors such as the system clock or program state. However, this approach is vulnerable to side-channel attacks [29], where malicious actors can manipulate the seed or influence the generated values. To address these vulnerabilities, the design incorporates a True Random Number Generator (TRNG) and a Pseudorandom Number Generator (PRNG), implemented directly in hardware to enhance security. As illustrated in Fig. 4, TRNG1 and TRNG2 utilize variations in circuit characteristics as sources of randomness, with slight environmental changes inducing unpredictable state alterations. The outputs of these TRNGs are combined using a bitwise XOR operation to generate seeds with high entropy for the PRNG. The generated seeds are temporarily stored in FIFO1, a data buffer. The PRNG retrieves seeds from FIFO1 to produce a sequence of random numbers, which are then stored in FIFO2 for future use. To ensure efficiency and immediate availability, the generator continuously fills FIFO2 with random numbers until the buffer reaches capacity. This design allows downstream modules to access random numbers on demand without waiting for the generation process to complete, ensuring seamless and secure operation.

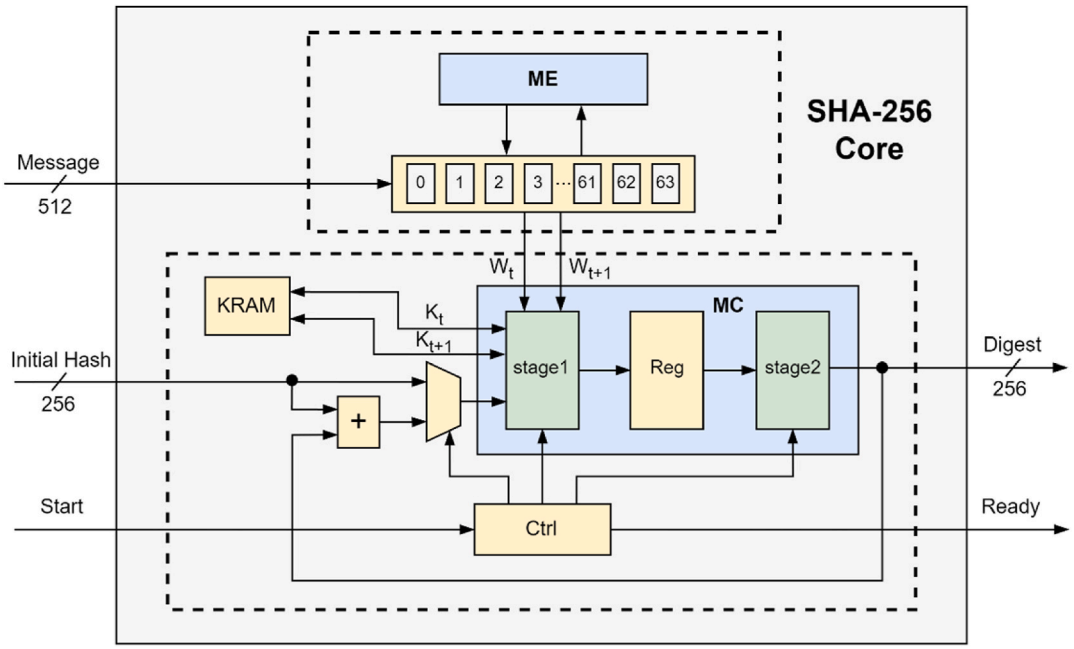


Fig. 5. Architecture of a SHA-256 unit.

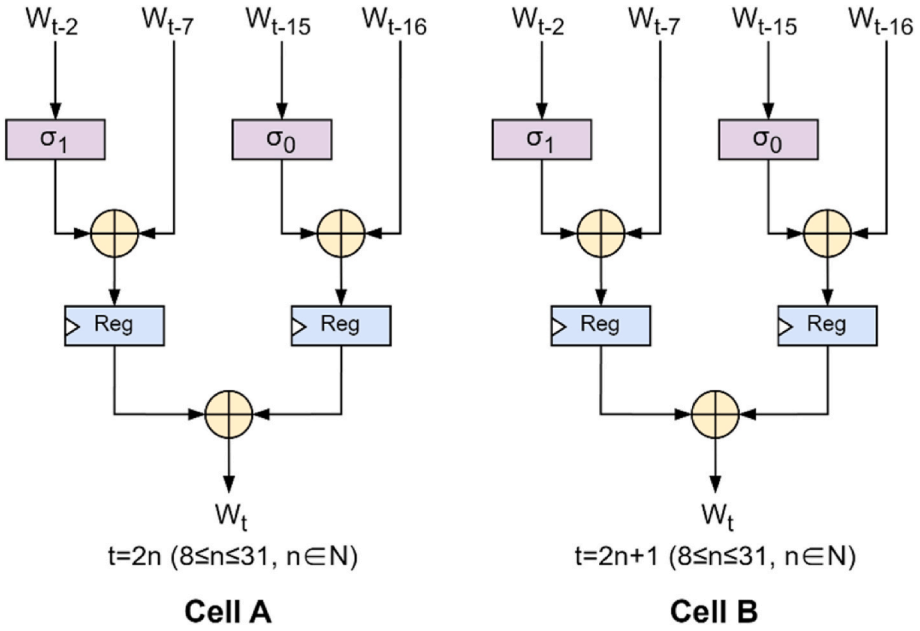


Fig. 6. Structure of an ME circuit.

3.2.2. SHA-256 computation unit

As depicted in Fig. 5, the SHA-256 computation unit is structured into two distinct components, corresponding to the message expansion and compression stages of the algorithm. The message expansion circuit consists of an ME unit and a register set. The register set contains 64 registers, each 32 bits wide and indexed from 0 to 63, to store the expanded words W_t. For small datasets, this register set supports efficient parallel read and write operations through multiple ports. The 512-bit input is divided into $W_0 \sim W_{15}$, which are stored in registers 0 to 15. Fig. 6 details the internal design of the ME unit, which includes two identical and independent components labeled cell A and cell B. These cells operate in a pipelined manner to reduce the critical path length and enhance processing speed. Cell A calculates W_t for even values of t, while cell B computes W_t for odd values. By working in parallel, these cells significantly improve computational efficiency. This design leverages the independence of data required for adjacent t values. For example, calculating W_{16} requires W_0, W_1, W_9, and W_{14}, whereas W_{17} depends on W_1, W_2, W_{10}, and W_{15}. Since the data dependencies do not overlap, W_{16} and W_{17} can be computed simultaneously, optimizing the message expansion process and accelerating the overall computation.

The message compression circuit is composed of three main components: KRAM, the MC unit, and a controller. KRAM serves as dual-port RAM to store the constants K_t necessary for the computation process. The controller manages data flow and monitors the computation's progress. The MC unit represents the core of the message compression circuit, executing the majority of operations required for SHA-256. Under typical conditions, the MC unit completes 64 iterations to compute the final values of the state variables $A \sim H$, consuming most of

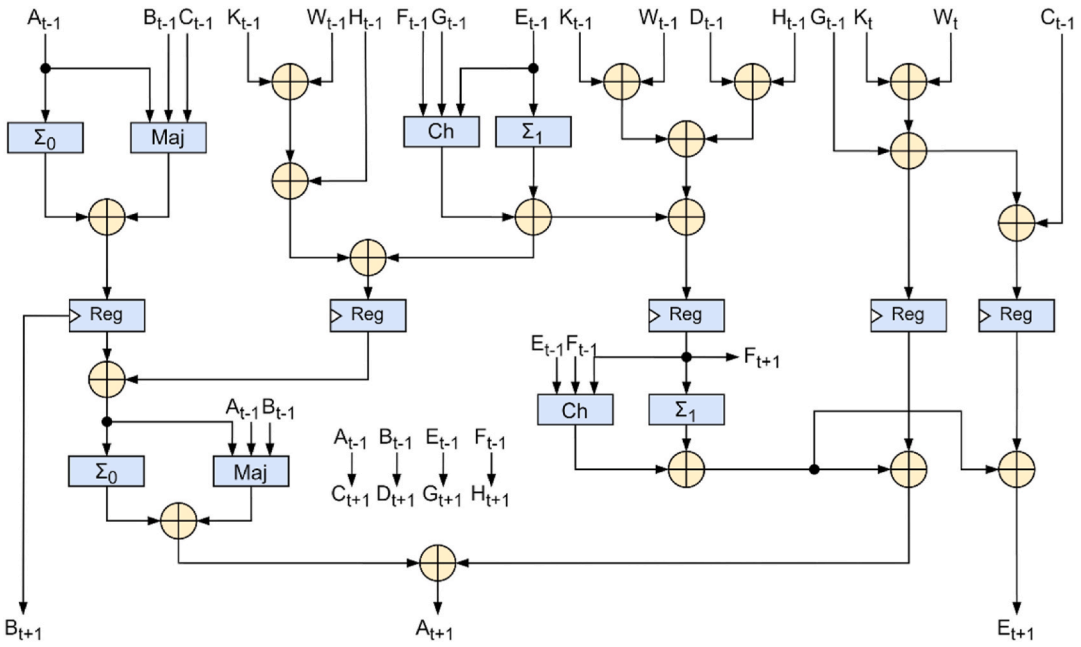


Fig. 7. Structure of an MC circuit.

the processing time. To improve efficiency, the circuit structure illustrated in Fig. 7 is employed to reduce the compression time. Conventionally, each MC operation derives $A_t \sim H_t$ (for $1 \leq t \leq 63$) from $A_{t-1} \sim H_{t-1}$, K_{t-1}, and W_{t-1}. However, leveraging the recursive relationship between state variable updates, the computation can be optimized. Specifically, $A_t \sim H_t$ can act as intermediate values to calculate $A_{t+1} \sim H_{t+1}$. By modifying the input to include K_t and W_t, it becomes possible to directly determine $A_{t+1} \sim H_{t+1}$ using $A_{t-1} \sim H_{t-1}$. This refinement reduces the number of state variable update operations to just 32 iterations, effectively halving the computation time for this stage. Furthermore, the ME unit is designed to compute W_{t-1} and W_t simultaneously. This enables the message expansion and message compression circuits to operate in parallel, significantly enhancing overall processing efficiency.

While this approach effectively halves the clock cycles needed for MC execution, it also increases the length of the combinational logic chain. The longest logic chain arises from the operations required to update the state variable A. In the standard implementation, this chain comprises four adders and one Σ_0 operation block in series. By contrast, the improved design involves six adders and one Σ_0 operation block, extending the critical delay path. To address this issue, registers are introduced within the combinational logic to divide the longest delay chain into smaller segments. This pipelining strategy ensures that the critical path is significantly shortened. After implementing the pipeline scheme, the longest logic chain now includes only three adders and one Σ_0 operation block, effectively balancing the trade-off between reduced clock cycles and increased combinational logic complexity.

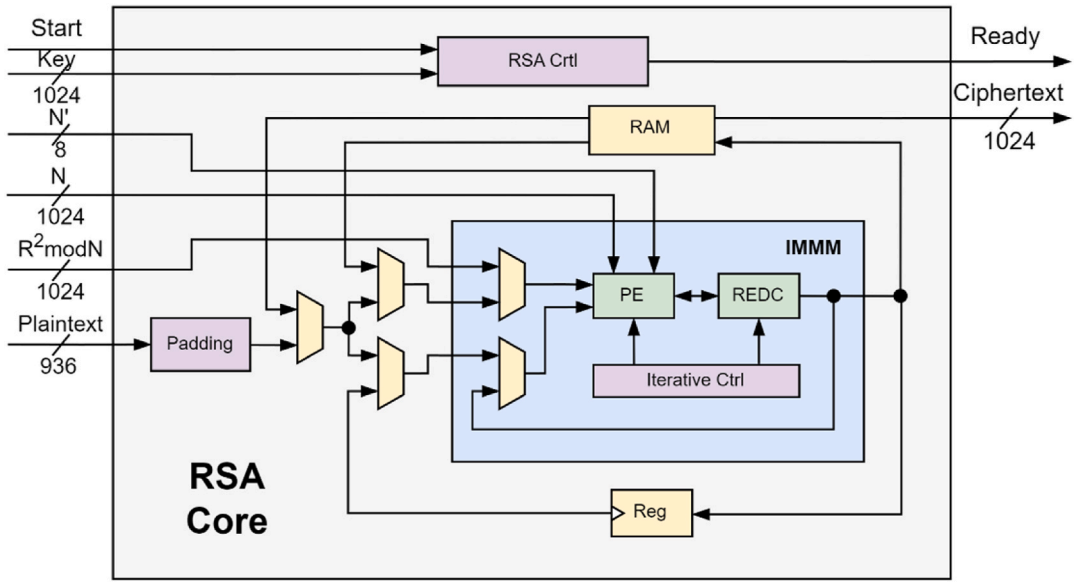


Fig. 8. Architecture of the RSA encryption engine.

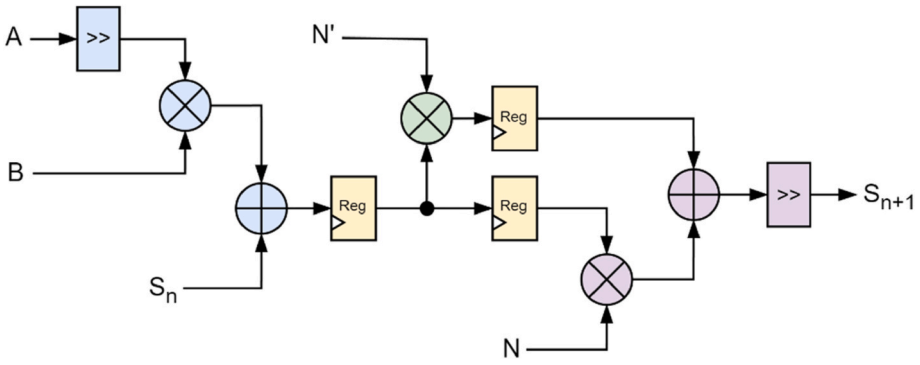


Fig. 9. Structure of the PE circuit.

3.2.3. RSA encryption engine

RSA encryption and digital signing fundamentally rely on modular exponentiation, which is executed through a series of modular multiplication operations, typically optimized using fast exponentiation algorithms. However, the modulo operation involved in modular multiplication poses challenges for hardware implementation. Efficient modular multiplication algorithms are thus vital to enhancing the performance of hardware accelerators. The Montgomery algorithm is a widely adopted method to address this issue, as it substitutes division operations with more hardware-efficient multiplication and shift operations [30]. Despite this improvement, the traditional Montgomery algorithm introduces large-width multiplications, which demand significant hardware resources. To overcome these limitations, this study presents a hardware accelerator designed around a base 2^k iterative Montgomery algorithm, detailed in Algorithm 1. The proposed approach segments lengthy multiplicands into smaller partitions and processes them iteratively to compute the result. In Algorithm 1, A and B represent the multiplicand and multiplier, N is the modulus, N' is a precomputed constant, and n denotes the number of partitions, equating to the iteration count. The algorithm produces an output scaled by a factor of 2^{-kn} relative to the correct result. To eliminate this scaling bias, a correction step is introduced: let $R = 2^{kn}$, and perform an additional Montgomery modular multiplication using the output and the precomputed constant $R^2 \bmod 2^k$ under the same modulus N. This step ensures the final result matches the expected value.

Algorithm 1. Radix-2^k iterative Montgomery modular multiplication

Input: integer A, B, odd N, $N' = (-N)^{-1} \bmod 2^k$, $n = |N|/k$, $A = \sum_{i=0}^{n-1} a_i 2^{ki}$
Output: $S_n = AB \cdot 2^{-kn} \bmod N$
1: $S_0 = 0$
2: **for** $i = 0$ **to** $n-1$ **do**
3: $S_{i+1} = S_i + a_i B$;
4: $q_i = S_{i+1} N'$;
5: $S_{i+1} = (S_{i+1} + q_i N) \gg k$;
6: **if** $S_{i+1} \geq N$ **then**
7: $S_{i+1} = S_{i+1} - N$;
8: **end**
9: **end**
10: **if** $S_n \geq N$ **then**
11: $S_n = S_n - N$;
12: **end**
13: **return** S_n

The structure of the RSA encryption engine is depicted in Fig. 8. To improve security, the Padding module applies padding to the input plaintext. The outer controller and RAM are responsible for executing fast exponentiation, with the controller calculating the necessary number of modular squaring steps according to the provided key and managing the data flow. Intermediate results are stored temporarily in the RAM. The IMMM module is tasked with performing modular multiplication operations. It comprises the PE unit, the Reduction module, and the Iteration controller. The PE unit includes a shifter, an adder, and a multiplier, which handles the core operations of the algorithm. The

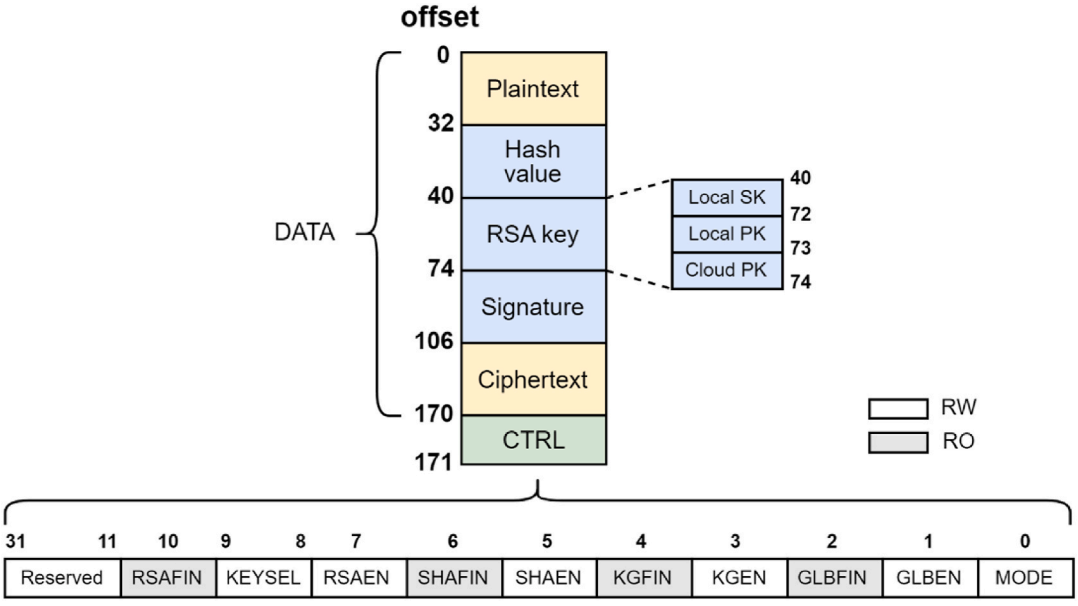


Fig. 10. Coprocessor memory mapping.

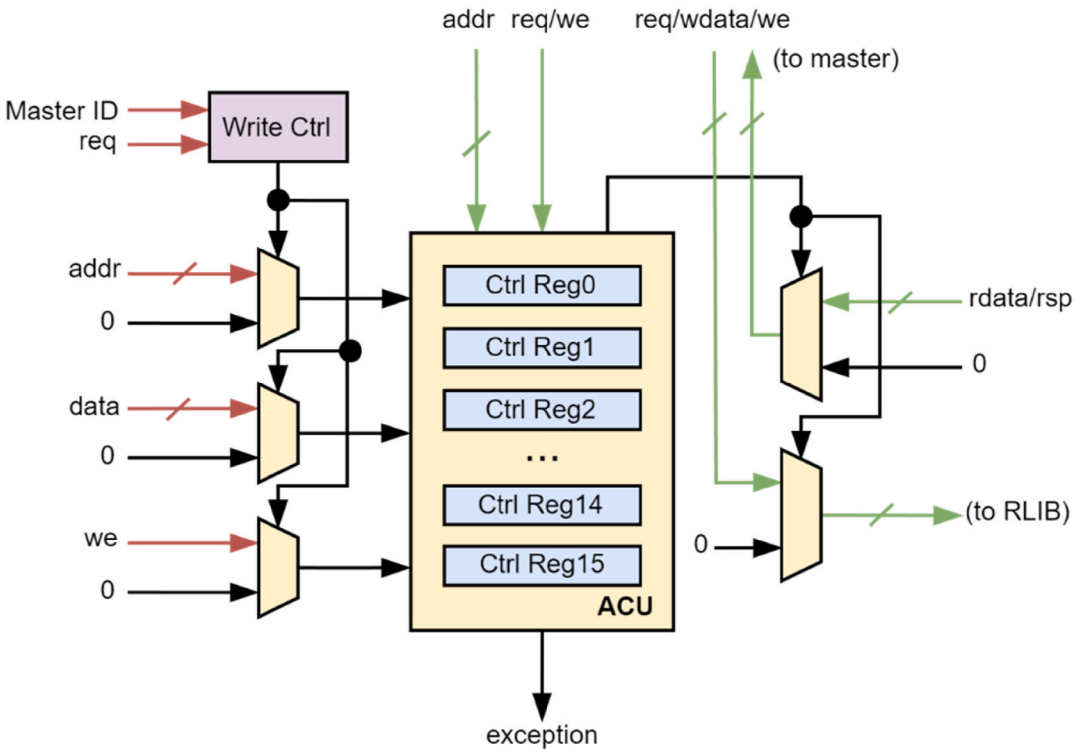


Fig. 11. Structure of the IOPMP circuit.

circuit structure of the PE is illustrated in Fig. 9. This design employs 2^8 as the base, and the multiplier blocks for each iteration are fetched through the shifter. To achieve a higher clock frequency, the PE is organized into a three-stage pipeline structure based on the operations performed during each iteration. The precomputed constants needed for the PE are derived by reusing the divider and extended Euclidean operation circuits from the key generation unit, optimizing resource usage. Once the PE completes its calculations, the results are passed to the REDC module to ensure the outputs are within the designated modulus range. The Iteration controller is in charge of managing data flow within the IMMM and controlling the number of iterations required.

3.2.4. System level configuration of coprocessor

The coprocessor is integrated into the SoC through memory mapping, enabling the RISC-V core to configure it and exchange data via standard memory access operations. As shown in Fig. 10, the coprocessor's data and control registers are mapped to a contiguous address space, referred to as the data segment and the control segment, respectively. The data segment includes plaintext, digest value, RSA keys, digital signature and ciphertext. Since the proposed SoC utilizes a bus with a maximum data width of 32 bits, data exceeding this width must be split and stored across multiple registers. The control segment includes a single 32-bit register that holds the configuration settings of the coprocessor. The bit definitions of the control register are also shown in Fig. 10.

The coprocessor integrates multiple encryption accelerators to meet the requirements of specialized tasks. By modifying the MODE bits in the control register, the coprocessor can operate in either ALL mode or SINGLE mode. In ALL mode, the RISC-V core simply feeds the plaintext into the coprocessor and asserts the global enable signal (GLBEN). The coprocessor then processes the plaintext following the data-on-chain flow, sequentially invoking all accelerators while indicating processing status via the GLBFIN signal. In this mode, individual accelerator configuration signals are ignored. Upon completion, the RISC-V core retrieves the ciphertext from the designated address space for subsequent transmission. In SINGLE mode, a specific accelerator is activated by setting its corresponding enable bit. When the processing status bit becomes valid, the RISC-V core accesses a predefined data segment region to obtain the result.

3.3. Trusted computing architecture

The fundamental concept of a trusted computing architecture involves partitioning the SoC into two distinct environments: a secure world and a normal world at the hardware level. Sensitive operations (such as blockchain-related tasks discussed in this paper) are performed in the secure world, while other tasks are executed in the normal world. This approach resembles a hardware-based firewall, where the IOPMP module sits between the main device in the normal world and the bus interface, preventing unauthorized access to secure world data. The circuit design is shown in Fig. 11, with red signal lines representing the slave interface and green signal lines representing the master interface. The Access Control Unit (ACU), a critical component of the IOPMP, is responsible for verifying the legitimacy of access operations to specific address ranges. It stores access permission details in its 16 internal 32-bit control registers, each linked to the address ranges of 16 peripherals on the RLIB L1 bus. Additionally, each address range is subdivided into 16 smaller intervals, with 2 bits in each register indicating access permissions: 00 for no access (read/write), 01 for no read but write access, 10 for read-only access, and 11 for both read and write access. Before any bus signals from non-secure master devices are transmitted, the ACU inspects them for validity. If the access is valid, the bus signals pass through the IOPMP normally. If the access is invalid, the signals are disregarded, and the ACU triggers an exception for the respective master device. In practice, this means that both the signals sent and received by the master device will be set to zero in case of invalid access. The ACU is configurable through the slave interface, and each master device within the SoC is assigned a unique ID. Upon receiving a write request, the IOPMP's internal write controller identifies the requester using this ID. Configuration of the ACU is reserved exclusively for the TEE core in the

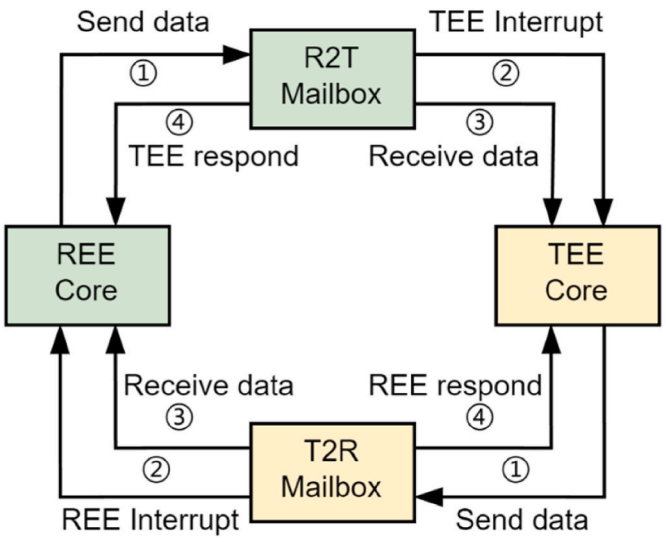


Fig. 12. Inter-core communication process via the Mailbox peripheral.

Table 1
Fpga resource utilization.

Module	LUTs	FFs	BRAMs	DSPs
REE Core	4785	1938	0	4
TEE Core	3354	1943	0	4
SHA-256	5548	2851	0	0
RSA Keygen	39146	16976	0	0
RSA	20387	15530	0	120
Complete Design	83471	44192	24	128

secure world, with requests from other master devices being disregarded.

The TEE core and REE core operate in separate memory spaces. Security programs are pre-programmed into the TEE ROM by developers, and these programs cannot be modified by users. On the other hand, the REE core allows users to access programming permissions, with user programs running in the REE ROM. To facilitate secure communication between the two cores, the proposed design utilizes the Mailbox peripheral. This Mailbox employs a FIFO buffer to store messages sent by the sender, with an internal status register that indicates the current state of the Mailbox, such as whether any data remains unread. The process of inter-core communication is illustrated in Fig. 12. The R2T Mailbox and T2R Mailbox support communication in both directions: from the REE core to the TEE core and vice versa, respectively. For example, when transmitting data from the REE core to the TEE core, the REE core writes the data into the FIFO buffer of the R2T Mailbox. Once the write operation is completed, the status register is updated to valid, and the R2T Mailbox triggers an interrupt signal to the TEE core, notifying it of the incoming data. Upon receipt of the data, the status register is reset to invalid, and an acknowledgment signal is sent back from the R2T Mailbox to the REE core, confirming the successful reception of the data. Both the R2T Mailbox and T2R Mailbox are independent of each other. To prevent unintended data leakage from the secure world, the T2R Mailbox is designated as a secure peripheral, ensuring that non-secure master devices are unable to access it.

4. Experimental results

4.1. FPGA prototyping and verification

The proposed SoC is implemented using Verilog HDL. To verify its functionality in a real hardware setting, a prototype of the design is deployed on the Kintex-7 series FPGA platform, with the chip model XC7K325TFFG676-2. The bitstream is generated using Xilinx Vivado 2022.2 and is transferred to the chip through the onboard JTAG interface. The FPGA resource usage is summarized in Table 1. Both the security and general programs are written in C language, with the security program pre-loaded into the TEE ROM, and the general program being loaded dynamically via a debugger connected to the REE core's debugging interface. To simulate data transmission from the edge device to the cloud server, the UART pins of the SoC are connected to a PC, allowing data to be sent via UART. The two CPU cores and the coprocessor of the SoC all operate at a clock speed of 50 MHz. The static power consumption, measured through Vivado, is 0.164 W, while the dynamic power consumption is 0.416 W, resulting in a total power consumption of 0.579 W. The verification results indicate that the proposed SoC successfully performs the data-on-chain processing task.

4.2. Performance evaluation

To evaluate the performance of the proposed SoC, this paper compares its execution time for various subtasks and the overall execution time of the data-on-chain processing task with that of an embedded MCU based on a RISC-V core and the high-performance general-purpose CPU (Intel i9-13950HX). Both the embedded MCU and the proposed SoC utilize the same RISC-V core and are implemented on the Kintex-7 FPGA platform. While the MCU and the i9-13950HX execute purely software-based programs, the SoC employs a hardware-software co-design approach, where the software invokes the coprocessor to perform specific functions. The test results are presented in Table 2.

SHA-256 is utilized to generate the hash value. When compared to the RISC-V MCU and the i9-13950HX, the SoC achieves a speed improvement of 26.2x and 2.2x, respectively. The execution time of the SoC is closer to that of the i9-13950HX because in the SoC, the data padding for SHA-256 is handled by software, which consumes a significant portion of the overall execution time.

The RSA-related tests are conducted under the worst-case scenario. For RSA encryption, the public exponent is set to the commonly used value of 65537, the modulus is a random 1024-bit value, and the plaintext is also 1024 bits. The results show that the RISC-V MCU takes 63.97 s to complete the encryption, with the primary time consumption arising from the modular exponentiation of large numbers in RSA encryption. Because the operand bit width exceeds the maximum bit width that can be handled by C language variables, arrays are needed to store the operands. These large-number operations are executed in multiple word-by-word steps, which are then broken down into a significant number of basic instructions. The low frequency of the MCU leads to high computational delays. Despite pre-calculated values and some optimizations, the results remain suboptimal. In contrast, the i9-

Table 2
Task execution speed in different environments.

Task	RISC-V MCU (Kintex-7)		Intel i9-13950HX (ASIC 10 nm)		Proposed SoC (Kintex-7)		Speedup (vs RISC-V MCU)	Speedup (vs Intel i9-13950HX)
	Freq.	Exec. Time	Freq.	Exec. Time	Freq.	Exec. Time		
SHA-256	50 MHz	1.31 ms	2.2 GHz	0.11 ms	50 MHz	0.05 ms	26.2	2.2
RSA Encrypt		63.97 s		127.25 ms		16.51 ms	3874.6	7.7
RSA Sign		66.90 s		312.84 ms		24.27 ms	2756.5	12.9
Total		254.71 s		687.36 ms		56.98 ms	4470.2	12.1

Table 3
Comparison of key algorithm accelerators.

Research	Algorithm	FPGA Platform	Freq. (MHz)	LUT	FF	Throughput (Mbps)	Area Efficiency (Mbps/1kLUT)
[34]	SHA-256	Cyclone II	87.08	10918	6131	655.66	60.05
[35]		Zynq 7000	181	6367	25190	917	144.02
[36]		Virtex XCV400	77	1306	-	308	235.83
This work		**Kintex-7**	**50**	**5548**	**2851**	**775.76**	139.83
[32]	RSA 1024-bits	Kintex-7	25	26546	10707	0.034	0.0013
[33]		Artix-7	90.9	-	–	0.037	–
[37]		Virtex-6	42.45	46362	18548	0.161	0.0035
This work		**Kintex-7**	**50**	**20387**	**15530**	**0.042**	0.0021

13950HX, with its high operating frequency and instruction set optimizations for complex mathematical operations, can complete the encryption in less time. On the other hand, the proposed SoC leverages a custom coprocessor for computation, completing the encryption in just 16.51 ms at a 50 MHz frequency. This represents a speed improvement of 3874.6x and 7.7x over the two pure software implementations. RSA signing, the most time-consuming part of the task, also benefits from the proposed SoC, with speed improvements of 2756.5x and 12.9x over the other two implementations, in the worst-case scenario where the private exponent, modulus, and plaintext are all 1024 bits.

The test results for the complete task indicate that the RISC-V MCU takes 254.71 s to complete the task. As noted earlier, the limited computational power of the low-power embedded MCU results in unacceptable latency when deploying blockchain-related algorithms purely in software. The proposed SoC achieves a speedup of 4470.2x and 12.1x compared to the RISC-V MCU and i9-13950HX, respectively. More importantly, the proposed SoC's power consumption is significantly lower than that of the i9-13950HX, making it a more suitable option for IoT devices.

4.3. Comparison with related works

Several studies have proposed SoCs that incorporate security algorithm accelerators. For instance, Huynh et al. [31] developed an SoC based on an ARM processor capable of performing digital signature functions to improve the security of IoT systems. It includes SHA-256 and RSA 1024 IP cores that operate independently. Furthermore, Karmakar et al. [32] focused on digital signatures but featured only an IP core that accelerates modular exponentiation, with the processor based on the RISC-V architecture's Rocket core. Mondal et al. [33] developed an RSA encryption unit using systolic Montgomery multiplication, which is integrated with a MicroBlaze processor to form an SoC. In contrast, the proposed SoC utilizes a trusted computing architecture with dual RISC-V cores. The design of the security coprocessor is compact, integrating a random number generator and various accelerator cores, tailored for data-on-chain processing. As a result, this design offers higher security and is better suited for IoT blockchain applications.

In terms of performance, as few SoCs are customized for the complete data-on-chain processing, this paper compares the performance of the proposed SoC in key data-on-chain algorithms with other relevant works, as shown in Table 3. For the SHA-256 algorithm [34], used a pipelined architecture and optimized adder designs for its accelerator. However, it has relatively low throughput, and demonstrates merely about 43 % of the area efficiency achieved by the proposed design. The accelerator proposed in Ref. [35] is developed using High-Level Synthesis (HLS). Although it achieves higher throughput compared to this design, its power consumption reaches 0.1 W, which is significantly higher than the 0.018 W observed for the SHA-256 component in this design. In terms of area efficiency, both designs exhibit comparable results. The design in Ref. [36] has higher area efficiency but lower absolute performance, with a throughput of only about 40 % of that of this design. For 1024-bit RSA operations, the proposed design improves area efficiency by 0.6 times compared to Ref. [32]. The implementation in Ref. [33] operates at a higher frequency, but its throughput is still lower [37]. trades area for performance, achieving higher area efficiency. However, it clearly consumes excessive resources. Overall, the key components of the security coprocessor in the proposed design exhibit high efficiency while maintaining a balanced trade-off among power, performance and area.

5. Conclusion

This paper presents a dedicated SoC equipped with a security coprocessor, designed to provide data-on-chain processing capabilities for edge IoT devices, facilitating the creation of cloud-edge collaborative IoT blockchain systems. The coprocessor design includes a combined random number generator architecture, integrating a TRNG and PRNG, to generate high-quality random numbers. The mathematical process of SHA-256 is optimized to reduce the execution cycles of the acceleration unit. Additionally, an RSA encryption engine is developed based on efficient modular exponentiation algorithms, and the RSA key generation unit is reused to generate the necessary constants for encryption, thereby reducing resource consumption. Additionally, to protect against direct infiltration attempts aimed at stealing keys or sensitive data, the design incorporates a trusted computing architecture based on dual RISC-V cores. Unauthorized access is effectively intercepted by the IOPMP module, and inter-core communication is facilitated through the Mailbox peripheral. The proposed design was prototyped and tested on an FPGA platform. The performance results demonstrate that, for complete data-on-chain operations, the SoC utilizing a hardware-software co-design approach achieves speed improvements of 4470.2x and 12.1x over the RISC-V MCU and Intel i9-13950HX CPU, respectively, both of which run pure software programs. The SoC's power consumption is low, with a static power consumption of 0.164 W, dynamic power consumption of 0.416 W, and a total power consumption of 0.579 W.

Future work will focus on optimizing the SoC further to reduce its area and transitioning the design to ASIC technology, thereby enhancing its widespread applicability. This design aims to foster deeper integration between IoT and blockchain technologies, helping to address data security concerns for users.

CRediT authorship contribution statement

Wentao Xu: Writing – review & editing, Writing – original draft. **Xiang Li:** Writing – review & editing, Writing – original draft. **Qiuyan Xu:** Writing – review & editing, Writing – original draft. **Yuxuan Zhang:** Writing – review & editing, Writing – original draft. **Xiang Wu:** Writing – review & editing, Writing – original draft. **Weilin Li:** Writing – review & editing, Writing – original draft. **Ruize Wang:** Writing – review & editing, Writing – original draft. **Gang Liang:** Validation. **Hao Guo:** Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

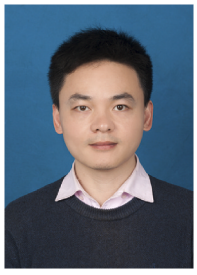
This work was supported in part by the Basic Research Programs of Taicang (TC2021JC24) and the Key R&D Programs of Taicang (TC2024SF10).

Data availability

Data will be made available on request.

References

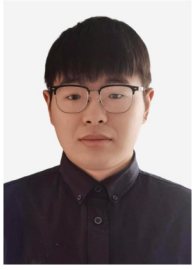
[1] E. Sisinni, A. Saifullah, S. Han, U. Jennehag, M. Gidlund, Industrial internet of Things: challenges, opportunities, and direc tions, IEEE Trans. Ind. Inf. 14 (2018) 4724–4734, https://doi.org/10.1109/TII.2018.2852491.
[2] M. Ammar, G. Russello, B. Crispo, Internet of Things: a survey on the security of IoT frameworks, J. Inf. Secur. Appl. 38 (2018) 8–27, https://doi.org/10.1016/j.jisa.2017.11.002.
[3] R. Zhou, L. Wang, J. Yang, Z. Li, X. Zhao, S. Liu, A 8.1-nW, 4.22-kHz, −40–85 °C relaxation oscillator with subthreshold leakage current compensation and forward body bias buffer for low power IoT applications, Microelectron. J. 144 (2024) 106090, https://doi.org/10.1016/j.mejo.2024.106090.
[4] N.M. Karie, N.M. Sahri, P. Haskell-Dowland, IoT threat detection advances, challenges and future directions, in: 2020 Workshop on Emerging Technologies for Security in IoT (ETSecIoT), April 2020. Sydney, NSW, Australia, 21-21.
[5] C. Huang, D. Liu, J. Ni, R. Lu, X. Shen, Achieving accountable and efficient data sharing in industrial Internet of Things, IEEE Trans. Ind. Inf. 17 (2021) 1416–1427, https://doi.org/10.1109/TII.2020.2982942.
[6] Z. Zheng, S. Xie, H. Dai, X. Chen, H. Wang, An overview of blockchain technology: architecture, consensus, and future trends, in: 2017 IEEE International Congress on Big Data (BigData Congress), 25-30 June 2017. Honolulu, HI, USA.
[7] P.D. DeVries, An analysis of cryptocurrency, bitcoin, and the future, Int. J. Bus. Manage. Commerce 1 (2016) 19.
[8] K. Christidis, M. Devetsikiotis, Blockchains and smart contracts for the internet of Things, IEEE Access 4 (2016) 2292–2303, https://doi.org/10.1109/ACCESS.2016.2566339.
[9] T.T.A. Dinh, R. Liu, M. Zhang, G. Chen, B.C. Ooi, J. Wang, Untangling blockchain: a data processing view of blockchain systems, IEEE Trans. Knowl. Data Eng. 30 (2018) 1366–1385, https://doi.org/10.1109/TKDE.2017.2781227.
[10] R. Huo, S. Zeng, Z. Wang, J. Shang, W. Chen, T. Huang, S. Wang, F.R. Yu, Y. Liu, A comprehensive survey on blockchain in Industrial Internet of Things: motivations, research progresses, and future challenges, IEEE Commun. Surveys Tuts. 24 (2022) 88–122, https://doi.org/10.1109/COMST.2022.3141490.
[11] A. Dorri, S.S. Kanhere, R. Jurdak, Towards an optimized BlockChain for IoT. Proceedings of the Second International Conference on Internet-Of-Things Design and Implementation, 2017, pp. 173–178, https://doi.org/10.1145/3054977.305500.
[12] M. Samaniego, R. Deters, Blockchain as a service for IoT, in: 2016 IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data, SmartData), Chengdu, China, December 2016, pp. 15–18.
[13] Y. Chen, L. Wang, S. Wang, Stochastic blockchain for IoT data integrity, IEEE Trans. Netw. Sci. Eng 7 (2020) 373–384, https://doi.org/10.1109/TNSE.2018.2887236.
[14] A. Dorri, S.S. Kanhere, R. Jurdak, P. Gauravaram, Blockchain for IoT security and privacy: the case study of a smart home, in: 2017 IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops), Kona, HI, USA, 2017.
[15] X. Zhang, R. Wu, M. Wang, L. Wang, A high-performance parallel computation hardware architecture in ASIC of SHA-256 hash, in: 2019 21st International Conference on Advanced Communication Technology (ICACT), PyeongChang, Korea (South), February 2019, pp. 17–20.
[16] I. Algredo-Badillo, C. Feregrino-Uribe, R. Cumplido, M. Morales-Sandoval, FPGA-based implementation alternatives for the inner loop of the secure hash algorithm SHA-256, Microprocess, Micro Syst. 37 (2013) 750757, https://doi.org/10.1016/j.micpro.2012.06.007.
[17] X. Huang, W. Wang, A novel and efficient design for an RSA cryptosystem with a very large key size, IEEE Trans. Circuits Syst. II, Express Briefs 62 (2015) 972–976, https://doi.org/10.1109/TCSII.2015.2458033.
[18] S.S. Ghoreishi, M.A. Pourmina, H. Bozorgi, M. Dousti, High speed RSA implementation based on modified Booth's technique and Montgomery's multiplication for FPGA platform, in: 2009 Second International Conference on Advances in Circuits, Electronics and Micro-electronics, Sliema, Malta, October 2009, pp. 11–16.
[19] W. Hentabli, F. Merazka, An extension of RSA_512 to RSA_1024 core under hardware platform based on montgomery powering, in: 2015 10th International Conference for Internet Technology and Secured Transactions (ICITST), December 2015, pp. 14–16. London, UK.
[20] P. Guo, Y. Yan, Z. Zhao, L. Zhang, C. Zhu, Z. Dai, R/B-SecArch: a strong isolated SoC architecture based on red/black concept for secure and efficient cryptographic services, Microelectron. J. 142 (2023) 106024, https://doi.org/10.1016/j.mejo.2023.106024.
[21] M. Gautschi, P.D. Schiavone, A. Traber, I. Loi, A. Pullini, D. Rossi, E. Flamand, F. K. Gürkaynak, L. Benini, Near-threshold RISC-V core with DSP extensions for scalable IoT endpoint devices, IEEE Trans. Very Large Scale Integr. Syst. 25 (2017) 2700–2713, https://doi.org/10.1109/TVLSI.2017.2654506.
[22] M.A. Ferrag, M. Derdour, M. Mukherjee, A. Derhab, L. Maglaras, H. Janicke, Blockchain technologies for the internet of Things: research issues and challenges, IEEE Internet Things J. 6 (2019) 2188–2204, https://doi.org/10.1109/JIOT.2018.2882794.
[23] H.N. Dai, Z. Zheng, Y. Zhang, Blockchain for internet of Things: a survey, IEEE Internet Things J. 6 (2019) 8076–8094, https://doi.org/10.1109/JIOT.2019.2920987.
[24] R. Yang, F.R. Yu, P. Si, Z. Yang, Y. Zhang, Integrated blockchain and edge computing systems: a survey, some research issues and chal lenges, IEEE Commun. Surveys Tuts. 21 (2019) 1508–1532, https://doi.org/10.1109/COMST.2019.2894727.
[25] M.S. Ali, M. Vecchio, M. Pincheira, K. Dolui, F. Antonelli, M.H. Rehmani, Applications of blockchains in the Internet of Things: a comprehensive survey, IEEE Commun. Surveys Tuts. 21 (2018) 1676–1717, https://doi.org/10.1109/COMST.2018.2886932.
[26] H.L. Pham, T.H. Tran, T.D. Phan, V.T.D. Le, D.K. Lam, Y. Nakashima, Double SHA-256 hardware architecture with compact message expander for Bitcoin mining, IEEE Access 8 (2020) 139634–139646, https://doi.org/10.1109/ACCESS.2020.3012581.
[27] A.A. Monrat, O. Schelén, K. Andersson, A survey of blockchain from the perspectives of applications, challenges, and opportunities, IEEE Access 7 (2019) 117134–117151, https://doi.org/10.1109/ACCESS.2019.2936094.
[28] R. Imam, Q.M. Areeb, A. Alturki, F. Anwer, Systematic and critical review of RSA based public key cryptographic schemes: past and present status, IEEE Access 9 (2021) 155949–155976, https://doi.org/10.1109/ACCESS.2021.3129224.
[29] J. Deogirikar, A. Vidhate, Security attacks in IoT: a survey, in: 2017 International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud) (I-SMAC), February 2017, pp. 10–11. Palladam, India.
[30] B. Zhang, Z. Cheng, M. Pedram, High-radix design of a scalable Montgomery modular multiplier with low latency, IEEE Trans. Comput. 71 (2022) 436–449, https://doi.org/10.1109/TC.2021.3052999.
[31] H.T. Huynh, T.K. Tran, T.P. Dang, T.T. Bui, Security enhancement for IoT systems based on SoC FPGA platforms, in: 2020 4th International Conference on Recent Advances in Signal Processing, Telecommunications & Computing (SigTelCom), August 2020, pp. 28–29. Hanoi, Vietnam.
[32] A. Karmakar, S. Sánchez-Solano, M.C. Martínez-Rodríguez, P. Brox, HW/SW implementation of RSA digital signature on a RISC-V-based System-on-Chip, in: 2023 38th Conference on Design of Circuits and Integrated Systems (DCIS), November 2023, pp. 15–17. Málaga, Spain.
[33] S. Mondal, S. Patkar, Hardware-Software co-implementation of a high performance and light-weight scalable Systolic-Montgomery based modified RSA for portable IoT devices, in: 2021 International Conference on Emerging Smart Computing and Informatics (ESCI), Pune, India, 2021.
[34] Z. He, L. Wu, X. Zhang, High-speed pipeline design for HMAC of SHA-256 with masking scheme, in: 2018 12th IEEE International Conference on Anti-counterfeiting, Security, and Identification (ASID), Xiamen, China, 2018.
[35] M. Kammoun, M. Elleuchi, M. Abid, M.S. BenSaleh, FPGA-based implementation of the SHA-256 hash algorithm, in: 2020 IEEE International Conference on Design & Test of Integrated Micro & Nano-Systems (DTS), 2020. Hammamet, Tunisia.
[36] R. Glabb, L. Imbert, G. Jullien, A. Tisserand, N. Veyrat-Charvillon, Multi-mode operator for SHA-2 hash functions, J. Syst. Architect. 53 (2–3) (2007) 127–138, https://doi.org/10.1016/j.sysarc.2006.09.006.
[37] V.R. K, S.S. C, V. Desalphine, D. Selvakumar, A low latency montgomery modular exponentiation, in: 2020 Third International Conference on Computing and Network Communications (CoCoNet'19), Procedia Computer Science, June 2020, https://doi.org/10.1016/j.procs.2020.04.087.



Wentao Xu received his Ph.D. degree in Integrated Circuit Science and Engineering from Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2020. Now, he is an Associate Researcher at the School of Microelectronics, Northwestern Polytechnical University. His research interests include digital integrated circuit design, design of specialized domain processors and FPGA application design.



Xiang Li received his B.S. degree in Microelectronic Science and Engineering from Northwestern Polytechnical University, Xi'an, China, in 2023. He is currently pursuing the M.S. degree in the School of Microelectronics, Northwestern Polytechnical University. His research interests focus on processor design and blockchain technology.



Qiuyan Xu received the B.S. degree in Microelectronics Science and Engineering from the School of Microelectronics, Northwestern Polytechnical University, Xi'an, China, in 2024. His research interests focus on blockchain data verification, optical convolutional neural network accelerators, and on-chip network design.



Yuxuan Zhang received the B.S. degree in Microelectronics Science and Engineering from School of Microelectronics, Northwestern Polytechnical University, Xi'an, China, in 2024, where he is currently pursuing the Master's degree. His research interests focus on digital integrated circuit design and AI hardware accelerator.



Xiang Wu received the B.S.degree in Microelectronics Science and Engineering from School of Microelectronics, Northwestern Polytechnical University,Xi'an,China,in 2024,where he is currently pursuing the Master's degree.His research interests focus on multi-core processors and DSP.



Weilin Li received the B.S. degree in Microelectronics from the School of Microelectronics, Northwestern Polytechnical University, Xi'an, China, in 2024. He is currently pursuing the M.S. degree at the same institution.His current research interests include machine learning, LSTM, and RISC-V architectures.



Ruize Wang received the B.S. degree in Microelectronics from the School of Microelectronics, Northwestern Polytechnical University, Xi'an, China, in 2019. He is currently pursuing the M.S. degree in Electronic Information Technology at the same university. His research interests include embedded systems and Transformer algorithm applications.



Gang Liang is currently pursuing a bachelor's degree in Microelectronics Science and Engineering at Northwestern Polytechnical University, Xi'an, China. He is particularly interested in the research and development of AI hardware accelerators and digital integrated circuits.



Hao Guo (Member, IEEE) received the B.S. degree from Northwest University, Xi'an, China, in 2012, and the M.S. degree from Illinois Institute of Technology, Chicago, IL, USA, in 2014, and the Ph.D. degree from the University of Delaware, Newark, DE, USA, in 2020, all in computer science. Currently, he is an Assistant Professor with the School of Software, Northwestern Polytechnical University, China. His research interests include blockchain and distributed ledger technology, privacy-preserving computation, applied cryptography, and Internet of Things (IoT) security. Dr. Guo is a Member of ACM and CCF.